Lab 4: CPU Decode + Control FSM, Program Counter and Associated Units

Raymond Tran, Robert Bulanon, Anna Larson, Dhiab Vaheedudheen

I. Introduction and CPU Overview

The objective of this lab was to design and implement a multicycle 16-bit CR16 CPU capable of fetching, decoding, and executing instructions stored in memory. The design integrates the ALU and register file developed in previous labs with a program counter, instruction register, instruction decoder, memory, and FSM controller. Together, these components support arithmetic and logic operations, memory access, conditional branches, and subroutine calls.

The CPU executes each instruction over several clock cycles. The FSM controller coordinates these stages by selecting data paths and enabling register, memory, and program counter updates. This approach allows instruction fetching and data access to share a single memory port while accommodating synchronous memory reads. The program counter identifies the current instruction address, and the instruction register holds the instruction stable during execution.

A test program loaded from a hexadecimal text file exercises the processor's operations, including loads and stores, arithmetic, loops, conditional branches, and subroutine calls. The testbench checks register and memory values against expected results. This CPU also provides the foundation for our proposed Dance Dance Revolution game, which will require keyboard input, a game timer, and a VGA display interface.

II. Datapath Design

The CPU datapath connects the register file, ALU, memory, program counter, and instruction register. The `cpu.v` module instantiates these components and connects them through data buses and multiplexers. Most data paths are 16 bits wide. Register addresses are 4 bits wide, allowing access to sixteen registers, and the RAM receives a 10-bit address for its 1,024 memory locations. The FSM controller coordinates execution by selecting data paths and enabling updates at the appropriate clock edges.

A. Instruction Fetch and Decode

During instruction fetching, the memory-address multiplexer selects the program counter as the memory address. The synchronous RAM returns the instruction on `mem_q`, and the instruction register captures it when `ir_en` is asserted. The instruction register preserves the instruction while the CPU completes its execution, including any subsequent data-memory access.

The instruction decoder extracts the register addresses, ALU operation, immediate value, branch displacement, and

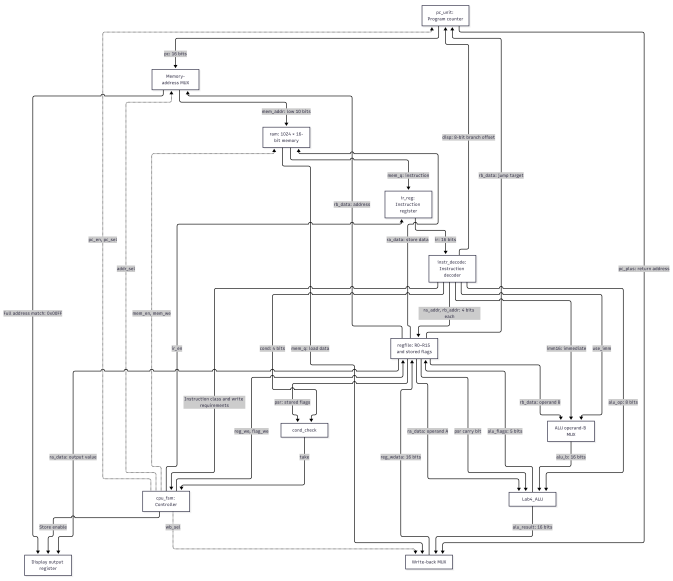


Fig. 1. CPU datapath and control connections. Solid arrows show data and decoded information, while dashed arrows show control signals. The memory-address multiplexer selects between instruction fetching and register-addressed data access. The write-back multiplexer selects an ALU result, loaded data, or a subroutine return address.

condition code. It also identifies the instruction type so that the controller can select the appropriate execution state. The register file provides two read outputs, `ra_data` and `rb_data`, corresponding to the selected registers.

B. ALU Operand Selection

The first ALU input connects directly to `ra_data`. The second input passes through a multiplexer controlled by `use_imm`. This multiplexer selects either `rb_data` for a register operation or `imm16` for an immediate operation. This allows the same ALU to execute both forms of an instruction.

The ALU produces a 16-bit result and five status flags. The controller separately enables result writes and flag writes. For example, an ADD instruction saves both its result and flags, while a CMP instruction updates the stored flags without overwriting a general-purpose register. The condition checker uses the stored flags to determine whether a conditional branch or jump is taken.

C. Memory Address and Store-Data Paths

The memory-address multiplexer is controlled by `addr_sel`. It selects `pc` during instruction fetching and `rb_data` during a load or store. The selected address

is carried by mem_addr, whose lowest ten bits connect to RAM.

Store data comes directly from ra_data. Therefore, a store uses register B to identify the memory location and register A to provide the value written there. The ALU is not part of the address or store-data path for these instructions. This provides a simple register-addressed memory interface. When a calculated address is needed, software can first compute the address and save it in a register.

D. Write-Back Selection

A three-input write-back multiplexer determines the value returned to the register file. The controller selects an ALU result for arithmetic and logic instructions, mem_q for a LOAD, or pc_plus for a jump-and-link instruction. The selected value appears on reg_wdata and is written to the register selected by ra_addr when reg_we is asserted.

Selecting pc_plus allows JAL to save the address of the following instruction in a link register while the program counter loads the jump target from rb_data. The saved address can later be used to return from the subroutine.

E. Example Datapath Operation

Consider ADD r3, r4, with R3 containing 5 and R4 containing 7. The decoder selects R4 for read port A and R3 for read port B. The operand multiplexer selects rb_data, so the ALU receives 7 and 5 and produces 12. During the execution state, the controller selects the ALU result for write-back and enables register and flag updates. At the rising clock edge, R4 receives 12, the flags are stored, and the program counter advances to the next instruction.

For LOAD r2, r1, the datapath instead uses the value in R1 as the memory address. After the synchronous read completes, the write-back multiplexer selects mem_q, and R2 captures theloaded value.

F. Display Output

The datapath also includes a 16-bit display output register. When a store targets address 0x00FF, the value on ra_data is copied into disp_out. The RAM write still occurs, so the value is retained in both memory and the output register.

III. FSM Controller Design

The cpu_fsm module controls instruction execution using a multicycle finite-state machine. The controller generates the enable signals and multiplexer selections that coordinate the datapath. Its outputs depend only on the current state, making it a Moore-style FSM. The next state depends on the current state, the decoded instruction type, and the condition checker's take signal.

The state register updates on each rising clock edge. When the active-low reset signal rst_n is asserted, the controller returns to S_FETCH. Reset is synchronous in this module.

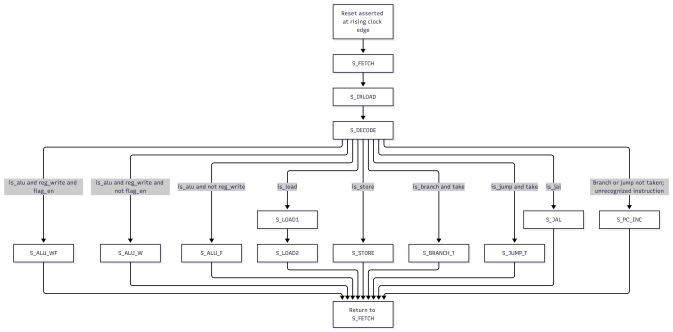


Fig. 2. CPU controller state-transition diagram. All instructions pass through fetch, instruction loading, and decoding before entering an execution state. LOAD requires two execution states to accommodate synchronous memory. After execution, the controller returns to instruction fetching.

A. Instruction Fetch and Decode

Every instruction begins with three states: S_FETCH, S_IRLOAD, and S_DECODE.

In S_FETCH, the controller asserts mem_en and selects the program counter as the memory address. The RAM performs a synchronous read, making the instruction available on its output after the rising clock edge.

In S_IRLOAD, the controller asserts ir_en. At the next rising clock edge, the instruction register captures the memory output. Separating these states allows the memory read to complete before the instruction is captured.

In S_DECODE, the decoder identifies the instruction type and selects the register operands. The controller does not enable any architectural writes during this state. At the next rising clock edge, it enters the appropriate execution state.

B. Arithmetic and Logic Execution

The controller uses three states for ALU instructions to distinguish which values must be saved.

- S_ALU_W: Writes the ALU result to the destination register without changing the stored flags. This state handles instructions such as MOV, logical operations, and shifts.
- S_ALU_WF: Writes the ALU result and updates the stored flags. This state handles the decoded ADD, ADDI, SUB, and SUBI instructions.
- S_ALU_F: Updates only the stored flags. CMP and CMPI use this state because their purpose is to compare operands without overwriting a general-purpose register.

Each of these states also enables the program counter to advance by one. The register, flag, and PC updates occur at the rising clock edge that returns the controller to S_FETCH.

C. Memory Access States

LOAD instructions use two execution states. In S_LOAD1, the controller selects rb_data as the memory address and enables a read. The synchronous RAM produces the requested data after the clock edge. In S_LOAD2, the controller selects memory data for write-back and asserts reg_we. The destination register captures the data, and the PC advances.

STORE instructions use S_STORE. In this state, rb_data supplies the address and ra_data supplies the value to store. The controller asserts mem_en and mem_we, causing the memory write at the rising clock edge. The PC advances at the same edge. No general-purpose register is written.

D. Branches and Subroutine Calls

For conditional branches and jumps, the condition checker evaluates the stored flags and produces take. The controller uses this result during S_DECODE.

A taken branch enters S_BRANCH_T, which selects the current PC plus the sign-extended displacement. A taken register-based jump enters S_JUMP_T, which selects rb_data as the new PC. If the condition is false, the controller enters S_PC_INC and advances to the following instruction.

JAL enters S_JAL. This state selects pc_plus for register write-back while selecting rb_data as the jump target. At the rising clock edge, the link register receives the return address and the PC receives the subroutine address.

Instructions not recognized by the decoder also enter S_PC_INC. The current implementation therefore advances past these encodings without performing a register or memory write.

E. Control Signals

The controller assigns default values to all outputs before applying state-specific changes. By default, write and update enables are disabled, the memory address selects the PC, write-back selects the ALU result, and the PC selection is sequential increment. These defaults prevent unintended updates and fully specify the combinational control outputs.

F. Execution Timing

Most supported instructions require four clock cycles: fetch, instruction loading, decode, and execution. LOAD requires five cycles because its execution separates the memory read from register write-back. After completing an instruction, the controller returns directly to S_FETCH.

For example, LOAD r2, r1 follows S_FETCH, S_IRLOAD, S_DECODE, S_LOAD1, and S_LOAD2. This sequence ensures that memory data is available before R2 captures it.

IV. FSM Testing Methodology

Verification uses two testbenches: tb_cpu.v tests the integrated processor, while tb_pc_unit.v tests the program counter independently. Both generate a 20 ns clock and count mismatches between expected and observed values.

A. Integrated CPU Testing

The CPU testbench loads lab4_program.hex into memory and allows the processor to execute the program through its normal fetch, decode, and execution stages. The program exercises arithmetic, logical operations, shifts, loads, stores, conditional branches, and subroutine calls. A repeated-addition loop computes 7×5, testing backward branches and termination based on updated flags.

The testbench checks final register and memory values. Examples include R8 containing 35 after the loop, memory location 0x00F2 containing 0xCDAB after a store, and R13 containing the return address 0x002E after JAL. Successful execution is expected to produce 0xCDCE in R0, memory location 0x00FF, and disp_out.

Execution is limited to 2,000 clock cycles while waiting for the completion address. The testbench also prints decoded instructions, register writes, memory writes, and flag updates to help identify incorrect behavior. Reset is checked before execution and asserted again after the program checks to verify that the PC returns to zero.

B. Program Counter Testing

The PC testbench applies control inputs on falling clock edges and checks results after the following rising edge. It tests sequential increments, the fallback increment selection, relative branches, absolute jumps, and pc_plus. Branch cases include offsets of $+5$, -1, $+127$, and -128, covering both directions and the limits of an 8-bit signed displacement.

Additional cases exercise 16-bit address wraparound and reset after the PC has changed. The expected value for an absolute jump must be the target register value, rather than the current PC plus the target. The jump expectation in the testbench requires this correction before its results can be used as verification evidence.

V. Program Counter Logic

The pc_unit module contains a dedicated 16-bit program counter, separate from the general-purpose registers. This preserves all sixteen registers for program data and keeps instruction-address updates within one module. The PC resets to zero and changes only when the controller asserts pc_en.

The CPU supports both relative and absolute control-flow addressing. Sequential execution increments the PC by one because each memory address holds one 16-bit instruction. Taken branches add a sign-extended 8-bit displacement to the current PC, while register-based jumps load an absolute target address from rb_data. The controller selects the update through pc_sel:

- 00: Increment the PC by one.
- 01: Add the signed branch displacement to the PC.
- 10: Load the target address from a register.

Relative branches provide a convenient way to implement nearby loops and conditional paths. Register-based jumps allow software to supply a target address and support subroutine returns.

For JAL, the CPU saves PC + 1 in the instruction's selected link register while updating the PC to the subroutine address. The demonstration program uses R13 as the return-address register, but the hardware does not reserve it exclusively for this purpose. A register-based jump through R13 returns to the calling program.

VI. Memory Implementation and FSM Interaction

The CPU uses the single-port synchronous RAM defined in `cpu.v`. It contains 1,024 locations, each holding 16 bits, for a total capacity of 2 KB. Instructions and data share this memory. The RAM receives the lowest ten bits of `mem_addr`, so software must keep accesses within the supported address range.

The controller selects the memory address through `addr_sel`. During instruction fetching, the address comes from the PC. During a LOAD or STORE, it comes directly from `rb_data`. Store data comes directly from `ra_data`; neither the address nor the stored value passes through the ALU. This keeps memory access simple. Software can calculate an address using the ALU beforehand and save it in a register when necessary.

Memory operations are coordinated with the FSM as follows:

- `S_FETCH`: Enables a read at the PC address. `S_IRLOAD` then captures the returned instruction.
- `S_LOAD1`: Enables a read at the address in register B. In `S_LOAD2`, the write-back multiplexer selects the memory output for the destination register.
- `S_STORE`: Asserts memory enable and write enable, storing register A's value at the address in register B.

Reads and writes occur on rising clock edges. Separating the read and capture stages ensures that memory output is available before the instruction or destination register captures it. When memory enable is low, the RAM contents and output remain unchanged. This sequencing allows instruction fetching and data access to share one memory port without conflicting accesses.

VII. Program Initialization and Instruction Encoding

The CPU loads its initial instructions and data from `lab4_program.hex` using `$readmemh`. The `INIT_FILE` parameter specifies the file to load. Each hexadecimal word represents a 16-bit instruction or data value. Address directives such as `@0F0` specify where subsequent words are placed in memory.

The corresponding assembly file, `lab4_program.asm`, provides a readable description of the program. The CPU executes the encoded words in the hexadecimal file. Changes to the assembly source must therefore be reflected in that file.

The demo exercises arithmetic, logical operations, shifts, loads, stores, conditional branches, and subroutine calls. It also computes 7×5 through repeated addition, testing backward branching and loop termination. Initial data values 7 and 5 are stored at addresses `0x00F0` and `0x00F1`.

Successful execution is expected to write `0xCDCE` to address `0x00FF`, which also updates `disp_out`. The program's failure routine writes `0xDEAD`. The testbench loads this actual program through the CPU's initialization parameter and checks the resulting register, memory, and output values.

VIII. Application I/O Requirements

Our DDR game will require a PS/2 keyboard interface, a game timer, and a VGA display controller. The keyboard interface will decode key presses and make input events available to the CPU. The timer will provide a consistent time reference for comparing key presses with expected note times. Software running on the CPU will determine hits and misses, update the score, and provide display information to the VGA controller. The VGA hardware will generate the video signals needed to display scrolling arrows and the current score.

We plan to connect these peripherals through memory-mapped registers, allowing software to access them using LOAD and STORE instructions. This requires address-decoding logic, selection between RAM and peripheral read data, and separate write enables to prevent I/O accesses from unintentionally modifying RAM. Keyboard events should remain available until the CPU acknowledges them, with timestamps captured in hardware to preserve their timing.

The existing `disp_out` register demonstrates a basic address-controlled output mechanism that can be extended for the game's peripherals.

IX. Timing, Area, and Resource Analysis

A. FPGA Resource Utilization

The current CPU implementation uses 11,856 ALMs, approximately 37% of the selected FPGA's available logic, and 16,791 registers. No dedicated memory blocks or DSP blocks were used.

Most resources are used by the RAM module currently instantiated inside `cpu.v`. The synthesis report assigns 16,400 registers and 6,926 combinational ALUTs to this module. This version does not connect the CPU to the dual-port memory developed in Lab 3.

The design fits on the FPGA, but integrating the BRAM-based memory implementation is expected to reduce logic and register usage.

B. Timing Analysis

The CPU was analyzed at a target frequency of 50 MHz, corresponding to a 20 ns clock period. The setup slack was 4.057 ns, and the hold slack was 0.467 ns. Both values are positive, so the analyzed paths meet the timing requirements. Quartus reported a maximum frequency of 62.72 MHz.

The most timing-critical path runs from the instruction register through register selection and memory-read logic to the memory output register. Its data delay was 15.294 ns, with approximately 76% coming from routing between components.

X. Conclusion

This lab integrated the ALU, register file, memory, and control logic into a multicycle 16-bit CPU. The FSM coordinates instruction fetching, decoding, and execution, while the program counter supports sequential execution, relative branches, and register-based jumps. A memory-loaded demonstration

program and testbench provide checks for arithmetic, memory access, conditional execution, and subroutine calls.

The resulting design provides a foundation for our DDR project. Future work will focus on connecting the keyboard interface, game timer, and VGA controller through memory-mapped I/O, then developing the software that handles note timing, hit detection, and scoring.